Data Types

While there are many data types available to the Arduino, for the moment we will start with the numerical types that use either whole numbers or floating-point numbers. Numeric data types are available in two varieties: **signed** and **unsigned**. Unsigned variables will always contain positive values, whereas signed variables can express numbers with a negative value. Usually most data types default to signed data types, but can be changed using the unsigned variable qualifier described later.

The **integer** data type is Arduino's primary variable data type for storing numbers; it can be used to store numerical values between -32,768 and 32,767. These values need to be whole numbers such as 42, 28,000, or -32, and can not include values with decimal points. Integer-type variables consume 16 bits or 2 bytes of our available 32 kilobytes of program memory—why this is important will become clearer when we discuss memory in Chapter 8. Integer variables can be declared with the **int** keyword, as follows:

```
int myVariable;
```

If you look at enough examples of Arduino code, you'll start to see that integers are a commonly used data type. For the most part, we will stick with this convention except where it makes more sense to choose another data type. Part of the reason for carefully choosing the proper data type for the job is that all numerical data types, including integers, will roll over in one direction or another when they exceed the minimum or maximum capacity. So given the following example:

```
x = 32767;
x = x + 1;
```

The variable `x` will now contain the value `-32,768`, as the integer exceeded its maximum capacity and rolled over to its minimum capacity and started again there. Remember that this also works both ways—so if `x = -32768;` and the following line is `x = x - 1;` then `x` will be equal to `32,767`.

As far as integers are concerned, there is only one value between 3 and 5, which is of course 4 because integers work in whole numbers. In contrast, **floating-point** numbers, expressed as the **float** data type, are capable of expressing fractions of a number using a decimal point and can be used to represent a multitude of values between 3 and 5, such as 3.14159, 4.25, or 5.0. Floats use 32 bits of memory and, like integers, can express values that are positive, negative, or zero. To declare a floating-point variable, we use the float keyword; and if assigning a value, we will also need to include a decimal point—even if the digit after the decimal point is 0. The following are two examples of float declarations:

```
float pi = 3.14159;
float circle = 360.0;
```

Because the Arduino lacks any dedicated hardware to manage floating-point math, floats are slow to process and may not yield the most accurate results. With only about 6 or 7 digits of precision, the expression z = 12.0 / 5.0 will yield 2.4000000953 instead of 2.4. While extremely helpful in certain situations, it is best to avoid floats where possible, especially in timing critical applications.

In addition to integers and floats, there are numerous other data types available on the Arduino. **Byte**, an 8-bit data type, can store values between 0 and 255. Unlike integers and floats, bytes are unsigned so that when a byte exceeds 255, it rolls over to 0. Bytes can be useful if we need to save some memory or are pretty sure we won't have a problem with data roll over.

```
byte dataPin = 11;
```

Other data types include the 32-bit **long**, which is the big brother of the integer, and the 1-bit **boolean** data type that only stores two values—either true or false. In Table 3-1 there are some of the various data types available to the Arduino platform, some of which we will cover in more depth later in the book.